

We introduce the first hierarchical method for large and complex featured (hyper)graph generation

Motivation

- Traditional one-shot approaches to graph generation struggle to scale effectively, which restricts the size and complexity of the graphs they can produce. This highlights the need for hierarchical methods.
- However, existing hierarchical approaches lack the ability to generate node and edge features, significantly limiting their applicability. Sequentially generating the topology followed by features is insufficient; a joint generation strategy is required.
- Hypergraphs—an extension of graphs in which *hyperedges* can connect more than two nodes—offer greater expressive power than standard graphs. For instance, they can naturally represent complex structures such as 3D meshes.

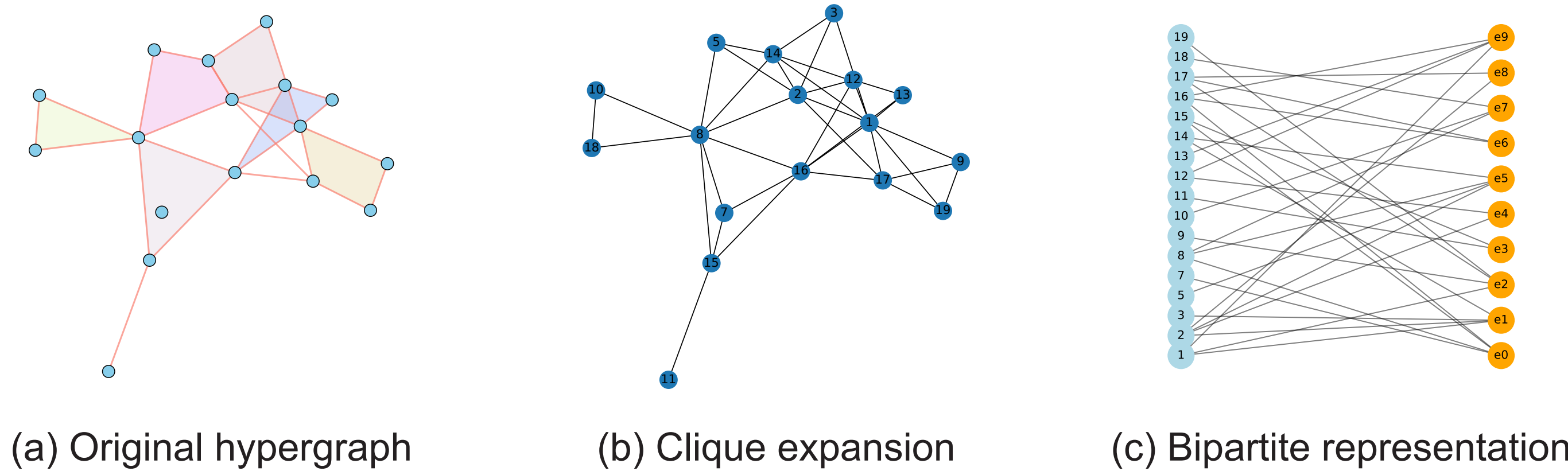
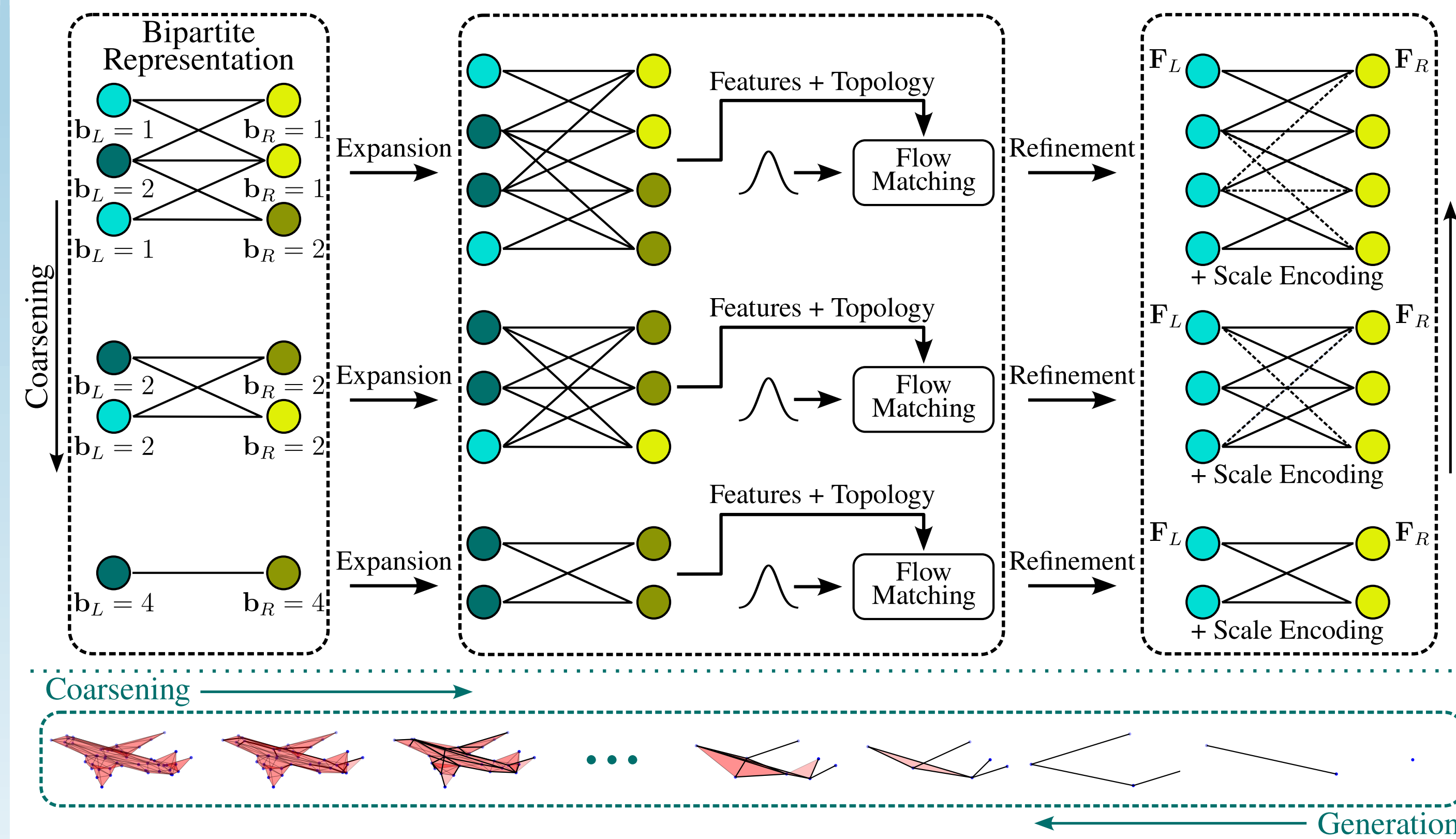


Figure. A hypergraph, its clique expansion and its bipartite representation

Pipeline



Like graphs [1] or images [2], unzoom (lose details) and train a model to zoom (reconstruct details). During training, hypergraphs are coarsened by merging nodes and hyperedges, averaging features and summing budgets. The model learns to predict merged nodes at each scale. In the expansion phase, those nodes are expanded with the same connectivity, feature and budget as their parents, and the model learns to: (a) identify excess edges, (b) split budgets, and (c) refine features.

Ideas

- Node budgets:** Rather than specifying a fixed number of nodes as input, the model learns to allocate *node budgets*—the number of nodes each coarse cluster should expand into—across different regions of the hypergraph during generation.
- Coarsening:** A coarsening algorithm [3] is applied to simplify the hypergraph by merging nodes while preserving overall connectivity. When nodes are merged, their features are averaged, and we record the number of original nodes within each cluster, *i.e.* *node budgets*.
- Generation:** The model is trained to reverse the coarsening process by expanding clusters back into individual nodes, inheriting their parent's connectivity, features, and budget. It then refines the structure by selectively pruning edges, updating features, and distributing each parent's budget among its children.

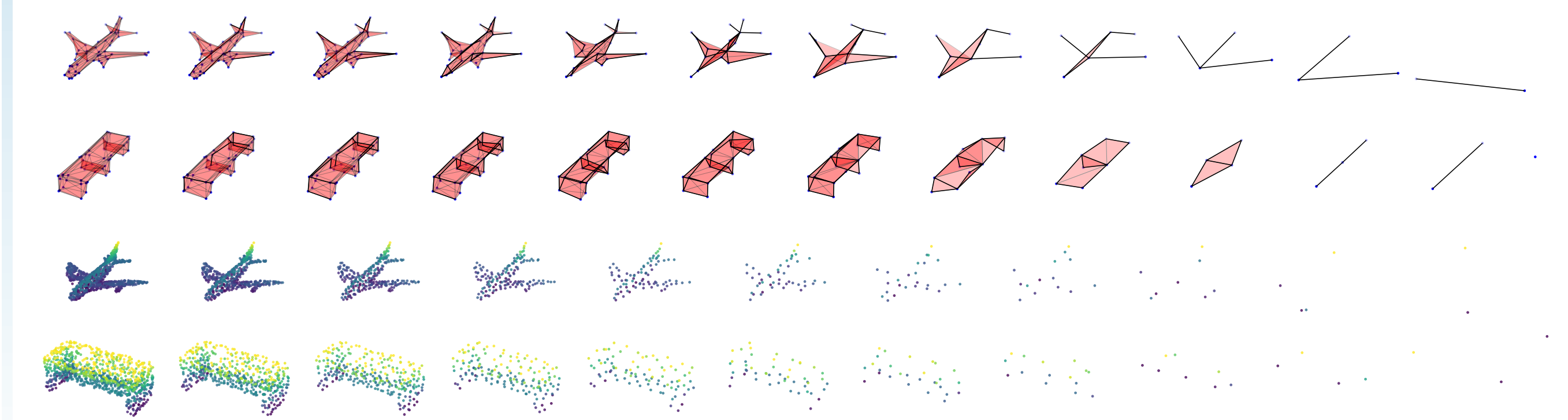
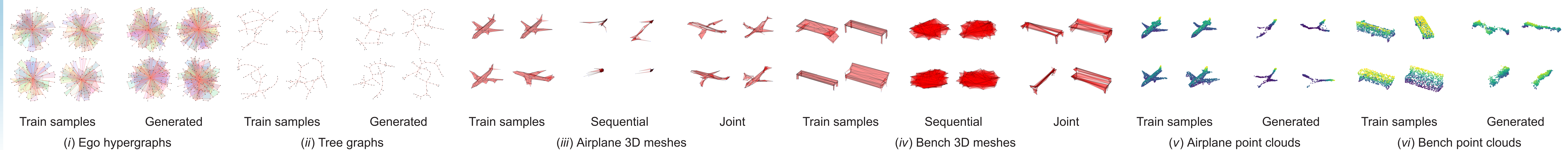


Figure. Examples of coarsening sequences for two 3D meshes and two point clouds.

Generated examples



Numerical Results

Method	Ego Hypergraphs		Method	Tree Graphs		Method	ManifoldNet40 Airplane			ManifoldNet40 Bench		
	Valid (%) $\uparrow$	Spectral $\downarrow$		Valid (%) $\uparrow$	Ratio $\downarrow$		ChamDist $\downarrow$	ChamCov (%) $\uparrow$	ChamDiv $\uparrow$	ChamDist $\downarrow$	ChamCov (%) $\uparrow$	ChamDiv $\uparrow$
HyperPA	0.0	0.237	HSpectre	100.0	4.0	Sequential	<u>0.082</u> $\pm$ 0.074	34.8 $\pm$ 0.0	<u>0.129</u> $\pm$ 0.011	0.169 $\pm$ 0.007	13.3 $\pm$ 0.0	0.157 $\pm$ 0.003
VAE	0.0	0.133	BwR	0.0	11.4	WGAN	0.124 $\pm$ 0.021	10.4 $\pm$ 3.5	0.037 $\pm$ 0.016	0.128 $\pm$ 0.015	17.3 $\pm$ 3.3	0.063 $\pm$ 0.099
GAN	0.0	0.230	DiGress	90.0	1.6	Flow Matching	0.085 $\pm$ 0.014	20.9 $\pm$ 10.0	0.063 $\pm$ 0.005	<u>0.108</u> $\pm$ 0.019	<u>30.7</u> $\pm$ 2.5	0.112 $\pm$ 0.007
Diffusion	0.0	0.190	DeFoG	96.5 $\pm$ 2.6	1.6 $\pm$ 0.4	VAE	0.231 $\pm$ 0.208	26.9 $\pm$ 11.7	0.191 $\pm$ 0.130	0.280 $\pm$ 0.223	27.5 $\pm$ 9.9	<u>0.195</u> $\pm$ 0.107
HYGENE	90.0	0.004										
FAHNES	99.5 $\pm$ 1.1	0.004 $\pm$ 0.003	FAHNES	100.0 $\pm$ 0.0	<u>1.8</u> $\pm$ 0.8	FAHNES	0.048 $\pm$ 0.003	<u>31.9</u> $\pm$ 13.2	0.101 $\pm$ 0.011	0.064 $\pm$ 0.005	42.2 $\pm$ 13.9	0.243 $\pm$ 0.037

Conclusions

- We propose a novel approach for hierarchical generation of featured (hyper)graphs.
- Our method draws inspiration from local expansion techniques [1] and the progressive upsampling strategy used in image generation [2].
- The introduction of the node budget mechanism and OT-coupling significantly enhances the quality of the generated topologies.

Next: Improve the robustness of the generation process.

[1] Bergmeister, A., Martinkus, K., Perraudin, N., Wattenhofer, R. (2023). *Efficient and scalable graph generation through iterative local expansion*

[2] S. Ren, Q. Yu, J. He, X. Shen, A. Yuille, L.-C. Chen. (2024). *FlowAR: Scale-wise autoregressive image generation meets flow matching*

[3] Loukas, A. (2019). *Graph reduction with spectral and cut guarantees*